Digital VLSI Design

Assignment Report

4-bit Array Multiplier from RTL to GDSII

with Open-Source Tools on SkyWater SKY130

Submitted by

Name	Roll number
M P Samartha	2023102038
Sai Abhinav Nagamalla	2024102037

International Institute of Information Technology, Hyderabad

October 2026

Technology	SkyWater SKY130 (`sky130A`), standard cells `sky130_fd_sc_hd`
Toolchain	IIC-OSIC-TOOLS 2026.10: Icarus Verilog, Yosys, OpenSTA, LibreLane/OpenROAD, Magic, KLayout, Netgen
Repository	https://github.com/NSA201006/sir-cute

Abstract

This report documents the complete open-source implementation of a 4-bit × 4-bit unsigned array multiplier, from Verilog RTL to a GDSII layout in the SkyWater SKY130 process. The design registers its inputs and its 8-bit product (16 flip-flops, 2-cycle latency) and builds the product from 16 AND gates and 12 full adders. An exhaustive self-checking testbench verifies all 256 input combinations (PASS: 256 vectors checked, 0 errors) and also checks that the latency is exactly two cycles. Yosys maps the design to 61 standard cells occupying 756.976 µm^2, of which the flip-flops take 52.89 %; the gate-level netlist reproduces the RTL behaviour (PASS: 256 vectors checked, 0 errors). Static timing analysis with OpenSTA gives a minimum clock period of 2.79 ns ($F_{\max}$ = 358.42 MHz). The critical path runs from register `a_r[1]` to register `p[6]` through 8 logic cells, which together realise eight full-adder stages, consistent with the $2N = 8$ full-adder-delay estimate. LibreLane takes the RTL to a 22500 µm^2 layout at a clock period of 3.5 ns with 0 DRC and 0 LVS errors and a setup slack of +0.519 ns, i.e. a post-route $F_{\max}$ of 335.44 MHz. All timing results are reported at the typical (TT) corner.

Contents

1 Introduction **3**

1.1 Objective . . . 3

1.2 Design specification . . . 3

2 Design Flow Overview **3**

3 Part A: RTL Design and Verification **4**

3.1 Array multiplier architecture . . . 4

3.2 Verification results . . . 5

3.3 Worked example . . . 6

4 Part B: Logic Synthesis with Yosys **7**

4.1 Procedure . . . 7

4.2 Results . . . 7

5 Part C: Static Timing Analysis with OpenSTA **8**

5.1 Procedure . . . 8

5.2 Results . . . 9

5.3 Critical path . . . 9

6 Part D: Physical Design with LibreLane **10**

6.1 Procedure . . . 10

6.2 Results . . . 11

6.3 Cells added by place and route . . . 12

6.4 Layout . . . 13

6.5 Post-route $F_{\max}$ versus Part C . . . 13

7 Answers to the Assignment Questions **13**

8 Conclusion **14**

9 Individual Contributions **15**

References **15**

1 Introduction

1.1 Objective

The objective of this assignment is to design a 4-bit unsigned array multiplier, verify it, and take it through a complete open-source RTL-to-GDSII flow on the SkyWater SKY130 130 nm process using the `sky130_fd_sc_hd` standard-cell library. The work is organised in four parts, which also structure this report:

Part A RTL design in Verilog and exhaustive verification with Icarus Verilog (Section 3).

Part B Logic synthesis to SKY130 standard cells with Yosys, followed by gate-level simulation (Section 4).

Part C Static timing analysis of the synthesized netlist with OpenSTA (Section 5).

Part D Physical design with LibreLane: floorplan, placement, clock-tree synthesis, routing, extraction, DRC, LVS and GDSII (Section 6).

All timing results are reported at the typical (TT) process corner: 25 °C and 1.80 V.

1.2 Design specification

Table 1 lists the interface of the top-level module `mult_array`. The multiplier registers both 4-bit operands and the 8-bit product, giving exactly 16 flip-flops. Counting clock cycles as the intervals between rising clock edges, an operand pair presented during cycle 0 is sampled into the input registers at the rising edge that ends cycle 0. The product is computed during cycle 1 and sampled into the output register at the next rising edge, so it is available at the output during cycle 2. The latency is therefore two clock cycles. A new operand pair can be accepted in every cycle, so the throughput is one product per cycle.

Table 1: Ports of `mult_array`.

Port	Direction	Width	Description
`clk`	input	1	clock, rising edge
`rst_n`	input	1	asynchronous reset, active low; clears all registers to 0
`a`	input	4	multiplicand
`b`	input	4	multiplier
`p`	output	8	product $a \times b$, registered

2 Design Flow Overview

Figure 1 shows the flow. The RTL is verified by simulation (Part A), synthesized to SKY130 standard cells (Part B), and the resulting netlist is timed with static timing analysis (Part C). LibreLane then takes the same RTL through the complete physical-design flow to a GDSII layout (Part D), using a clock period derived from the minimum period found in Part C.

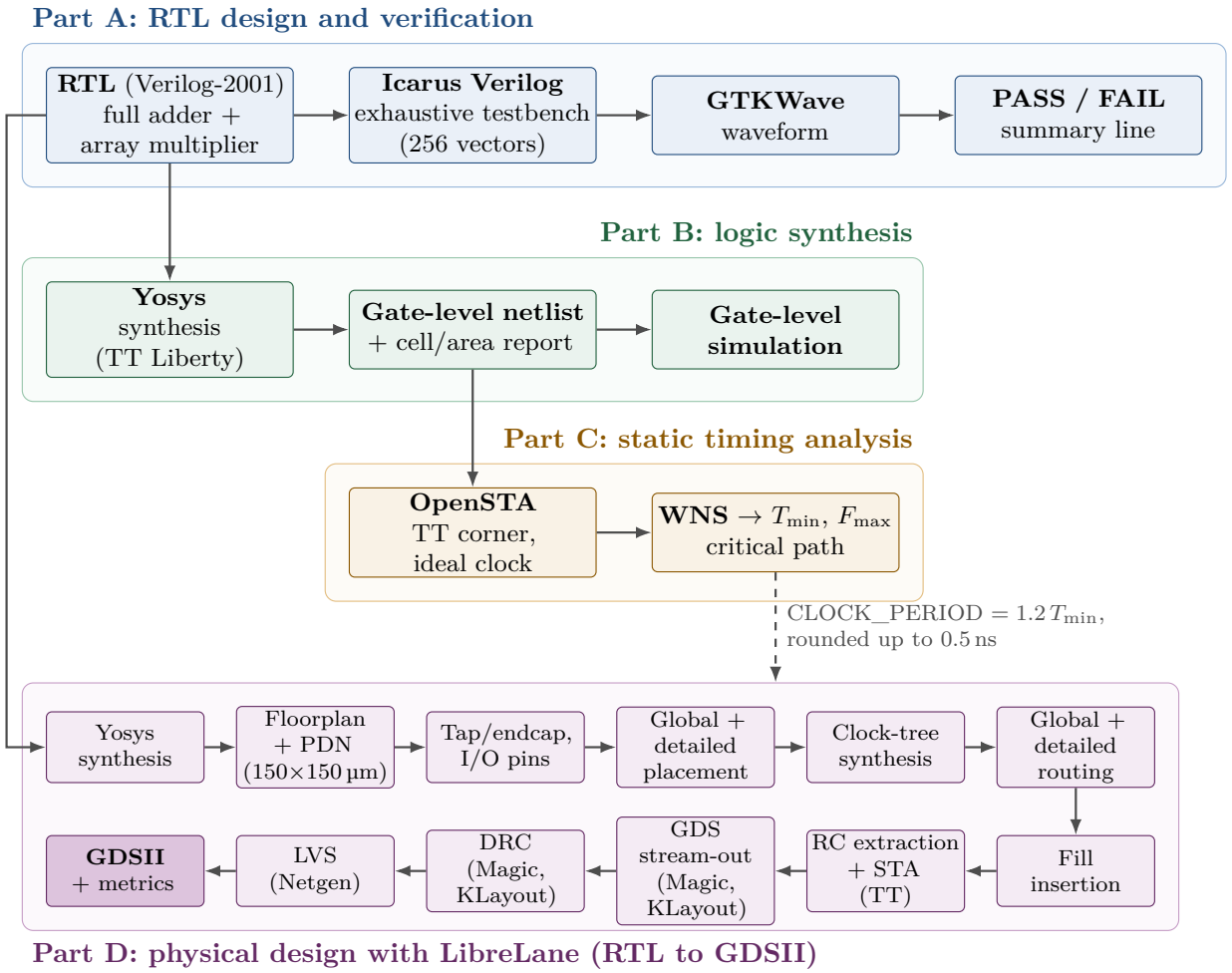


Figure 1: RTL-to-GDSII flow used in this work.

3 Part A: RTL Design and Verification

3.1 Array multiplier architecture

Multiplying $A = a_3a_2a_1a_0$ by $B = b_3b_2b_1b_0$ is shift-and-add. Each multiplier bit b_i selects one partial product $pp_i[j] = a[j] \cdot b[i]$, weighted by 2^{i+j}, and the four partial products are summed:

$$P = \sum_{i=0}^{3}\sum_{j=0}^{3} pp_i[j]\,2^{i+j}, \qquad pp_i[j] = a[j] \wedge b[i]. \tag{1}$$

The array implementation adds the partial products one row at a time (assignment Section 2):

- **Row 0** consists only of AND gates. $p[0] = pp_0[0]$, and the remaining bits $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$ pass to row 1.
- **Rows 1–3** are 4-bit ripple-carry adders of four full adders each. Full adder j of row i adds $pp_i[j]$, $r_{i-1}[j]$ and the carry $c_i[j]$ (with $c_i[0] = 0$). Its lowest sum bit is product bit $p[i]$, and $r_i = \{c_i[4], s_i[3], s_i[2], s_i[1]\}$ passes down.
- Row 3 also produces the top four product bits, $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$.

In total the array uses $N^2 = 16$ AND gates and $N(N-1) = 12$ full adders for $N = 4$. Figure 2 shows the structure as implemented in the RTL, drawn so that each column holds one bit weight 2^{i+j}.

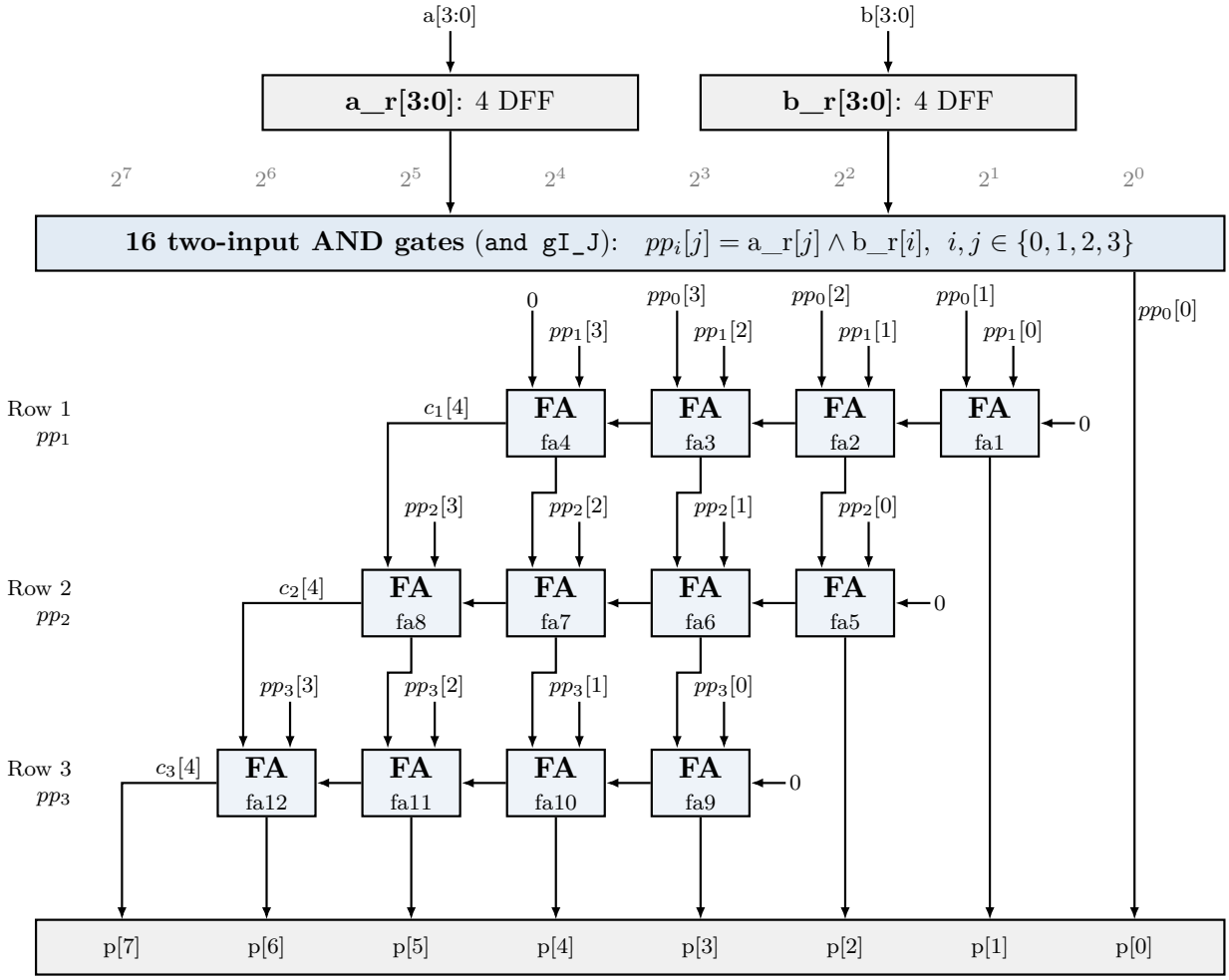


Figure 2: Structure of `mult_array` as implemented: input registers, 16 AND gates producing the partial products $pp_i[j]$, three rows of four full adders (instances `fa1`–`fa12`), and the output register. Columns are bit weights. Horizontal arrows are the ripple carries $c_i[j+1]$; vertical arrows pass the sum bits $s_i[j]$ to the next row (as $r_i[j-1]$) or into the output register. Partial-product inputs are labelled at each full adder; they come from the AND array.

3.2 Verification results

The self-checking testbench uses a 10 ns clock, holds `rst_n` low for the first two rising edges, and applies all 256 operand pairs on falling edges. For each pair it compares `p` with $a * b$ after the second rising edge, and it also checks that `p` has not yet changed after the first rising edge, so the two-cycle latency itself is verified. It prints a line for every mismatch, a single PASS/FAIL summary line at the end, and writes a waveform file.

Table 2: Part A verification results.

Check	Result
RTL simulation, 256 vectors	`PASS: 256 vectors checked, 0 errors`
Error-detection test (`sum = a ^ b`)	`FAIL: 256 vectors checked, 95 errors`

For the error-detection test the full adder's sum was temporarily changed to `a ^ b`. The first mismatch occurs at $a = 3$: for $a \in \{0, 1, 2\}$, a has at most one bit set, the partial products never

overlap, no carry is generated and the faulty sum is never exercised.

3.3 Worked example

Using the roll number of M P Samartha (2023102038), $A = 8 + 5 = 13$; since $A \neq 7$, $B = 7$. By hand, in the layout of the assignment:

```
          1 1 0 1      A = 13
        x 0 1 1 1      B = 7
  ----------------
          1 1 0 1      pp0 = A AND b0 (b0 = 1)
        1 1 0 1        pp1 = A AND b1 (b1 = 1), shifted 1
      1 1 0 1          pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
  ----------------
0 1 0 1 1 0 1 1      P = 91
```

Table 3 follows the same multiplication through the rows of the array (hand calculation with the equations of Section 3.1).

Table 3: Worked example 13×7 through the array rows (hand calculation).

Row	Addition $r_{i-1} + pp_i$	Sum s_i	$c_i[4]$	Product bit	Passed down
0	$pp_0 = 1101$	–	–	$p[0] = 1$	$r_0 = 0110$
1	$0110 + 1101 = 1\,0011$	0011	1	$p[1] = 1$	$r_1 = 1001$
2	$1001 + 1101 = 1\,0110$	0110	1	$p[2] = 0$	$r_2 = 1011$
3	$1011 + 0000 = 0\,1011$	1011	0	$p[3] = 1$	$p[7{:}4] = 0101$

Table 4 and Figure 3 show the same multiplication in the RTL simulation. Cycle 0 starts at the rising edge at 4315 ns (edge 0); during this cycle the testbench applies $a = 13$, $b = 7$ (at 4320 ns). Edge 1 at 4325 ns samples them into `a_r`, `b_r`, and during cycle 1 the combinational product is 91 while `p` still holds the previous product. Edge 2 at 4335 ns samples the product into `p`, which reads 91 during cycle 2. The operands presented in cycle 0 thus appear as the product in cycle 2, a latency of two clock cycles.

Table 4: Signal values of the worked example in the RTL simulation.

Time (ns)	Event	a	b	a_r	b_r	prod (comb.)	p
4320	cycle 0: operands applied	13	7	13	6	78	78
4325	edge 1: operands sampled	13	7	13	7	**91**	78
4335	edge 2: product sampled	13	7	13	7	91	**91**

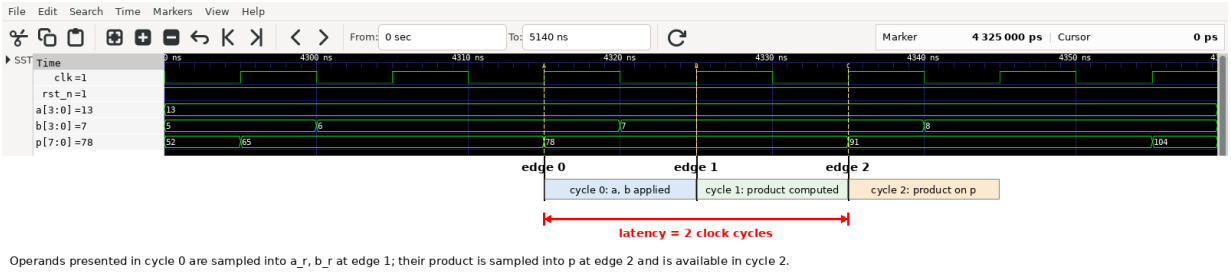


Figure 3: RTL simulation waveform (GTKWave) of the worked example with `clk`, `rst_n`, `a`, `b` and `p`. Markers A, B and C are the rising edges 0, 1 and 2 (4315, 4325 and 4335 ns). The operands are applied in cycle 0, sampled at edge 1 (`p` still shows $78 = 13 \times 6$ from the previous operands), and the product $\texttt{p} = 91$ is sampled at edge 2 and is available in cycle 2: a latency of two clock cycles.

4 Part B: Logic Synthesis with Yosys

4.1 Procedure

The Yosys script follows the seven steps prescribed by the assignment. It reads the RTL, runs generic synthesis with the full adders flattened into one module, maps the flip-flops and then the combinational logic to `sky130_fd_sc_hd` cells using the typical-corner (TT) Liberty file, removes unused wires, and writes the cell/area report and the gate-level netlist. The netlist was then simulated with the unchanged testbench and the SKY130 cell models. These functional models give the flip-flops a unit delay and the logic zero delay, so the gate-level simulation checks the function of the netlist, including the two-cycle latency, but not its timing (Part C).

4.2 Results

Table 5: Part B results (TT Liberty).

Metric	Value
Total number of cells	**61**
Number of flip-flops	**16** (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	**756.976** μm^2
Flip-flop area	**400.384** $\mu m^2 = 16 \times 25.024\,\mu m^2$
Flip-flop area (% of total)	**52.89 %**
3 most-used cell types	`dfrtp_1` $\times 16$; `nand2_1` $\times 7$; then a tie at 4: `and2_0` $\times 4$ and `maj3_1` $\times 4$
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 6: All 21 cell types in the synthesized netlist, sorted by count.

Cell (`sky130_fd_sc_hd__*`)	Count	Area (μm^2)
`dfrtp_1`	16	400.384
`nand2_1`	7	26.275
`and2_0`	4	25.024
`maj3_1`	4	40.038
`a21oi_1`	3	15.014
`xnor2_1`	3	26.275
`xor3_1`	3	71.318
`a22oi_1`	2	15.014
`clkinv_1`	2	7.507
`lpflow_isobufsrc_1`	2	12.512
`nand3_1`	2	10.010
`nand4_1`	2	12.512
`nor2_1`	2	7.507
`xor2_1`	2	17.517
`a21boi_0`	1	7.507
`and3_1`	1	6.256
`and4_1`	1	8.758
`o2111ai_1`	1	8.758
`o21bai_1`	1	7.507
`o311ai_0`	1	8.758
`xnor3_1`	1	22.522
Total	61	756.976

Observations.

- The netlist has 45 combinational cells for the 16 AND gates and 12 full adders (28 RTL primitives). The cells do not map one-to-one onto the RTL primitives (Section 7, Q1); in particular, synthesis cannot use the multi-output SKY130 full-adder cell.
- A flip-flop ($25.024\,\mu m^2$) is 3.2 times larger than the average combinational cell ($7.92\,\mu m^2$), so the 16 flip-flops account for 52.89 % of the area.

5 Part C: Static Timing Analysis with OpenSTA

5.1 Procedure

The OpenSTA script follows the steps prescribed by the assignment. It reads the TT Liberty file and the synthesized netlist, defines the clock with period T, applies 1.0 ns input and output delays, reports the worst setup slack and writes the critical-path report. It was run at $T = 10$ ns. The minimum period follows as $T_{\min} = T - \mathrm{WNS}$, rounded to 0.01 ns, and the analysis was repeated at $T = T_{\min}$ to confirm a slack close to zero.

5.2 Results

Table 7: Part C results (TT corner).

Metric	Value
Worst slack at $T = 10\,\text{ns}$	**7.21** ns (7.214832 ns at full precision)
$T_{\min}$	**2.79** ns ($= 10 - 7.214832 = 2.785168\,\text{ns}$, rounded to 0.01 ns)
$F_{\max} = 1000/T_{\min}$	**358.42** MHz
Slack at $T = T_{\min}$	**0.00** ns (+0.004832 ns at full precision, met)
Critical path start point	register `a_r[1]` (flip-flop `_91_`, `dfrtp_1`), rising edge of `clk`
Critical path end point	register `p[6]` (flip-flop `_88_`, `dfrtp_1`), setup check at the next rising edge
Number of cells on the critical path	**10**: launch flip-flop, 8 combinational cells, capture flip-flop
Latency $= 2 \times T_{\min}$	**5.58** ns
Throughput	**358.42** million results per second (one result per cycle)

5.3 Critical path

Table 8 lists the cells of the critical path in order, and Figure 4 shows the delay contribution of each cell. The path is the same at $T = 10\,\text{ns}$ and at $T = T_{\min}$, with a data arrival time of 2.650 ns.

Table 8: Critical path at the TT corner ($T = T_{\min}$). The last column gives the RTL full-adder signal that each cell output equals over all 256 input combinations (¬ = complement).

#	Instance	Cell	Incr (ns)	Arrival (ns)	Equals RTL signal
1	`_91_`	`dfrtp_1`	0.398	0.398	launch FF (CLK→Q)
2	`_37_`	`clkinv_1`	0.073	0.471	¬`a_r[1]`
3	`_41_`	`o311ai_0`	0.354	0.826	merged logic (no single RTL signal)
4	`_45_`	`a21oi_1`	0.152	0.978	fa5.cout
5	`_51_`	`maj3_1`	0.402	1.380	fa6.cout
6	`_58_`	`xnor3_1`	0.370	1.750	¬fa7.sum
7	`_61_`	`maj3_1`	0.368	2.117	¬fa10.cout
8	`_63_`	`maj3_1`	0.354	2.471	¬fa11.cout
9	`_64_`	`xor3_1`	0.178	2.650	fa12.sum
10	`_88_`	`dfrtp_1`	0.000	2.650	capture FF (D)
Library setup time of the capture flip-flop: 0.135 ns; required time $T - 0.135$ ns.					

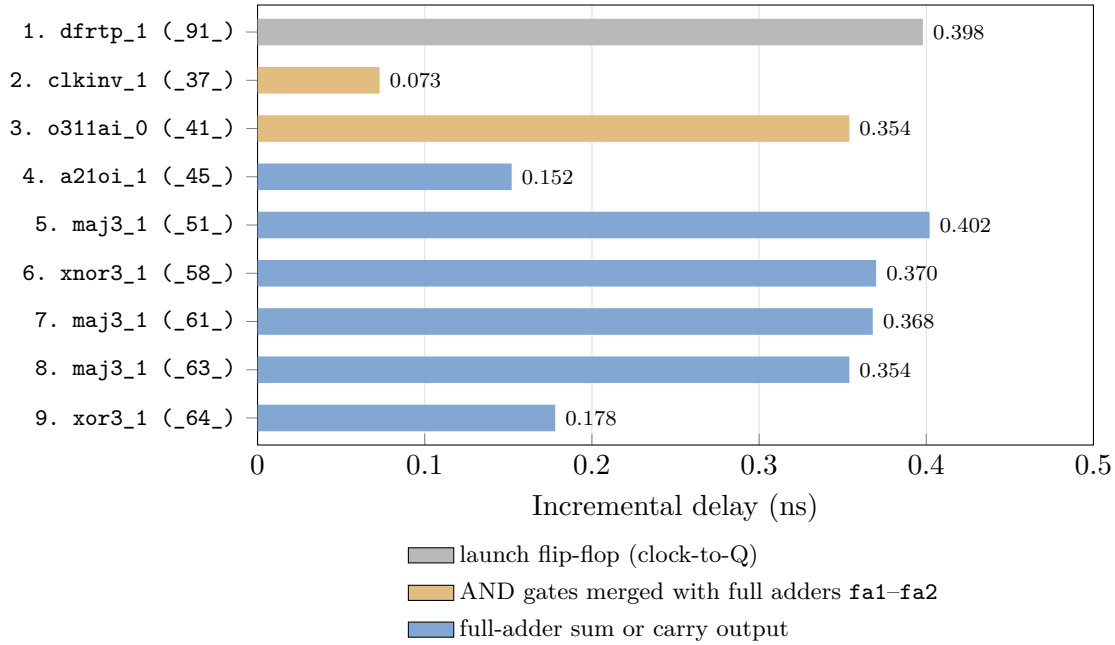


Figure 4: Delay contribution of each cell on the critical path, in path order (TT corner). The data arrival time at the capture flip-flop is 2.650 ns.

Full-adder count and the $2N$ estimate. Of the 8 combinational cells, 6 compute an exact full-adder sum or carry (`fa5`, `fa6`, `fa7`, `fa10`, `fa11`, `fa12`). The remaining two implement the partial-product AND gates together with the first two full adders (`fa1`, `fa2`), which synthesis merged. The path therefore crosses eight full-adder stages (Figure 5), which matches the estimate of about $2N = 8$ full-adder delays.

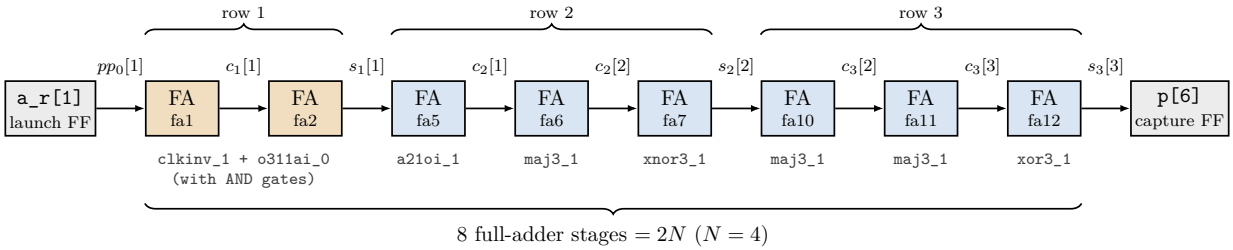


Figure 5: The critical path traced through the full adders of the array, with the cell that implements each stage. Horizontal labels give the signal passed between stages (c: carry, s: sum).

6 Part D: Physical Design with LibreLane

6.1 Procedure

LibreLane runs the complete RTL-to-GDSII flow of Figure 1. Starting again from the RTL, it performs synthesis, floorplanning with power-grid generation, tap/endcap insertion, I/O placement, global and detailed placement, clock-tree synthesis, timing repair, global and detailed routing, fill insertion, parasitic extraction, multi-corner static timing analysis, GDSII stream-out with Magic and KLayout, DRC with both tools, and LVS with Netgen. The configuration contains exactly the six keys required by the assignment:

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"
    ],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 3.5,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

The clock period is $1.2 \times T_{\min} = 1.2 \times 2.79 = 3.348\,\text{ns}$, rounded up to the next 0.5 ns, i.e. 3.5 ns. All other settings are LibreLane defaults; those relevant to timing are listed in Table 9 and are stricter than the Part C constraints.

Table 9: LibreLane default timing constraints compared with Part C.

Setting	LibreLane (Part D)	Part C
I/O delay	20 % of the clock period	1.0 ns
Output load	33.442 fF	none
Input driving cell	`inv_2`	ideal
Clock uncertainty	0.25 ns	none
Clock transition	0.15 ns	ideal
Timing derate	±5 %	none
Clock network	propagated (clock tree)	ideal
Wire parasitics	extracted (RCX)	none
Synthesis strategy	`AREA 0`	default `abc` script

Pass criterion. Consistent with Part C, timing is judged at the typical (TT) corner. The flow passes if it has zero DRC errors (Magic, KLayout and the router), zero LVS errors and a non-negative TT setup slack (TT hold slack was also checked).

6.2 Results

The first run, at 3.5 ns, met every criterion and is therefore the final run. LibreLane's default signoff additionally analyses slow and fast process corners; consistent with Part C, this report evaluates timing at the typical corner. At the slow-slow corners the default multi-corner signoff reports setup violations (worst −2.2565 ns at `max_ss_100C_1v60`, metrics key `timing__setup__ws`), so LibreLane exits with status 2; DRC, LVS and TT timing are clean.

Table 10: Part D results (TT corner).

Metric	Value
CLOCK_PERIOD used	**3.5** ns
Number of runs needed	**1** (all criteria met at 3.5 ns)
Core area	**17759.5** µm^2 (die area 22500 µm^2)
Cells after synthesis (Part B)	**61**
Cells after place and route	**1873**, of which 96 functional cells
Kinds of cells added	tap, decap, fill, clock buffers, I/O and repair buffers (Table 11)
Wirelength	**2610** µm
Setup slack (TT, `timing__setup__ws__corner:nom_tt_025C_1v80`)	+**0.5188** ns
Hold slack	+0.249 ns
Post-route $F_{\max} = 1000/(\text{CLOCK_PERIOD} - \text{slack})$	**335.44** MHz $= 1000/(3.5 - 0.5188)$
$F_{\max}$ from Part C	358.42 MHz
DRC errors	**0** (Magic 0, KLayout 0, router 0)
LVS errors	**0**
Setup / hold / max-slew / max-cap violations	0 / 0 / 0 / 0
Antenna-violating nets	0

6.3 Cells added by place and route

Table 11 compares the cell composition of the Part B netlist, LibreLane's own synthesis netlist and the final layout.

Table 11: Cell composition from synthesis to the final layout.

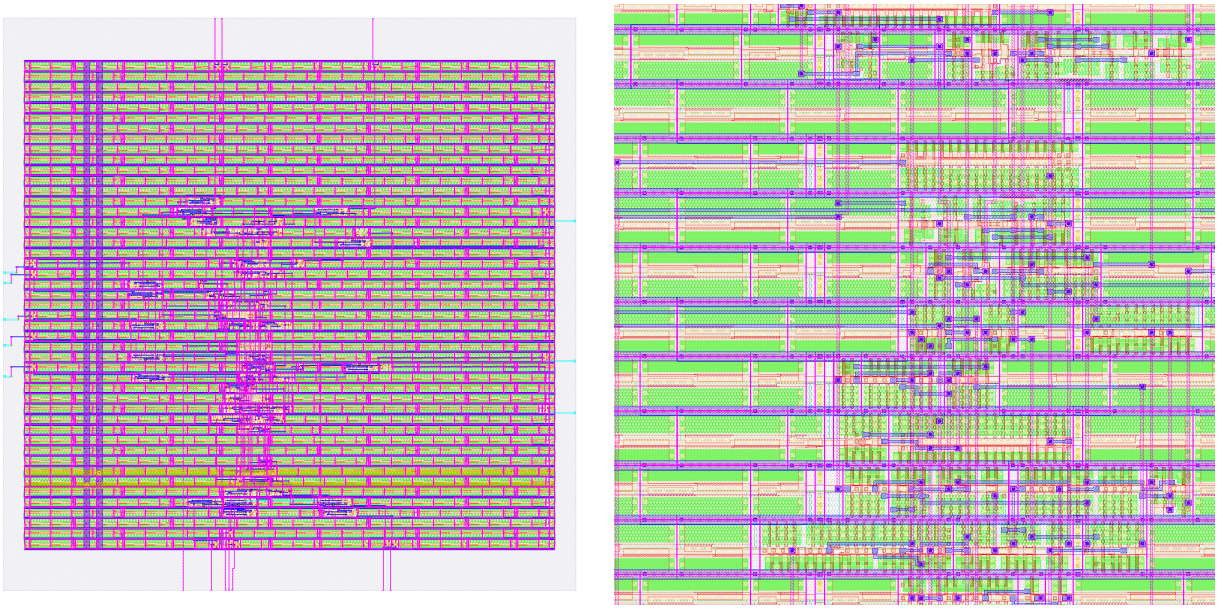
Cell kind	Part B netlist	LibreLane synthesis	Final layout
Flip-flops (`dfrtp`)	16	16	16
Logic cells	43	42	42
Inverters (`inv`/`clkinv`)	2	4	4
Buffers (`buf_*`: 9 input + 8 output + 12 rebuffer)	0	0	29
Fanout-repair buffers (`clkdlybuf*`)	0	0	2
Clock buffers (`clkbuf_*`)	0	0	3
Tap cells (`tapvpwrvgnd_1`)	0	0	245
Decap cells (`decap_*`)	0	0	1281
Fill cells (`fill_*`)	0	0	251
Total	61	62	1873

Logic cells exclude the 2 `clkinv_1` inverters, which are included in the 45 combinational cells of Section 4.2.

LibreLane re-synthesizes the RTL with its own script and excludes some cells that the Part B mapping used (`clkinv*`, `lpflow_*`), so its netlist has 62 cells (758.2272 µm^2) against 61 in Part B. During placement and clock-tree synthesis the resizer also upsized several logic cells (for example `maj3_1` to `maj3_4`) and 2 flip-flops (`dfrtp_1` to `dfrtp_2`).

6.4 Layout

Figure 6 shows the final layout. The die is $150 \times 150\,\mu\text{m}$; standard-cell rows fill the core, the power grid runs in vertical and horizontal straps, and the pins are placed on the die edges. The 96 functional cells occupy a small region near the centre, and the rest of the rows hold decap and fill cells.



(a) Whole die ($150 \times 150\,\mu\text{m}$). **(b)** Zoom on the logic: $x = 47$–$77\,\mu\text{m}$, $y = 42$–$72\,\mu\text{m}$.

Figure 6: Final layout (KLayout, SKY130 layer colours). The zoom shows standard-cell rows bounded by horizontal metal-1 power rails, cell geometry (poly, diffusion, local interconnect), and signal routing in metal 1 (horizontal) and metal 2 (vertical) with vias.

6.5 Post-route $F_{\max}$ versus Part C

At the TT corner the post-route $F_{\max}$ is 335.44 MHz, 6.4 % below the 358.42 MHz of Part C. The effective minimum period rises from the unrounded $T_{\min} = 2.785168\,\text{ns}$ to $3.5 - 0.5188 = 2.981\,\text{ns}$, an increase of 0.196 ns. Section 7 (Q4) discusses the causes.

7 Answers to the Assignment Questions

Q1. Cells for the logic, and why not $16 + 12$. Synthesis produced 61 cells, of which 16 are flip-flops, so the logic uses $61 - 16 = 45$ cells. The count is not $16 + 12$ because synthesis does not map the RTL gates one-to-one. The synthesis tool cannot use the multi-output SKY130 full-adder cell, so each full adder is built from several single-output cells. Flattening also lets it merge AND gates into complex gates (e.g. `a21oi`, `o311ai`) and simplify the full adders that have a constant input (`fa1`, `fa4`, `fa5`, `fa9`).

Q2. Critical path. The critical path starts at register `a_r[1]` (flip-flop `_91_`) and ends at register `p[6]` (flip-flop `_88_`, D input `fa12.sum`). It has 10 cells: the launch flip-flop, 8 combinational cells and the capture flip-flop (Table 8). Through the array, the path follows the chain `fa1`→`fa2`→`fa5`→`fa6`→`fa7`→`fa10`→`fa11`→`fa12`: eight full-adder stages, which agrees with the estimate of about $2N = 8$ full-adder delays (Figure 5). 6 of the 8 combinational cells compute an exact full-adder sum or carry; the first two implement the AND gates merged with `fa1` and `fa2`.

Q3. Flip-flop share of the area. The 16 flip-flops occupy 400.384 of 756.976 µm^2, i.e. 52.89 % of the cell area. A `dfrtp_1` cell (25.024 µm^2) is a complete master–slave flip-flop with clock buffering and an asynchronous reset. It is 3.2 times larger than the average combinational cell of this netlist (7.92 µm^2). The design is also very small: registering both operands and the product needs $4N = 16$ flip-flops, while the logic between them is only 45 cells. Since the logic grows as N^2 and the registers as $4N$, the flip-flop share falls for wider multipliers.

Q4. Why the post-route $F_{\max}$ is lower, and which cells were added. The post-route $F_{\max}$ is 335.44 MHz, 6.4 % below the 358.42 MHz of Part C. Part C assumed ideal wires and an ideal clock with no timing margins. After place and route, the timing also includes the delay of the routed wires, the clock tree, the I/O buffers, and LibreLane's default margins (0.25 ns clock uncertainty and ±5 % derating). The flow added (Table 11): **tap cells** (245), which tie the wells to the supply rails to prevent latch-up; **decap cells** (1281), which add decoupling capacitance to keep the supply stable; **fill cells** (251), which fill the gaps in the rows so wells and power rails stay continuous; **clock buffers** (3), which distribute the clock with low skew; and **buffers** at the I/O ports and for timing repair ($9 + 8$ port buffers, $12 + 2$ repair buffers).

Q5. DRC and LVS. A **design-rule check (DRC)** verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures and overlaps on every layer, density and antenna rules. Violations mean the masks may not print reliably, causing shorts, opens or low yield. A **layout-versus-schematic (LVS)** check extracts the devices and their connectivity from the layout and compares them with the reference netlist. It confirms that the layout implements exactly the verified circuit, with no missing or extra devices, no shorts and no opens. Both must pass before manufacture. A DRC-clean layout can still be the wrong circuit, and a circuit-correct layout that violates the rules may not be manufacturable. The final layout has 0 DRC errors in Magic, 0 in KLayout and 0 after routing, 0 LVS errors (Netgen) and 0 antenna-violating nets.

Q6. Scaling to 8 bits (estimate from the formulas of Section 2). For $N = 8$ the array needs $N^2 = 64$ AND gates (4× the 16 of the 4-bit design) and $N(N-1) = 56$ full adders (12 for $N = 4$, i.e. about 4.7×). The critical path grows to about $2N = 16$ full-adder delays instead of 8, so the combinational delay roughly doubles, growing linearly with N. Using the measured Part C path as calibration, the delay per full-adder stage is $(2.650 - 0.398)/8 = 0.281$ ns (data arrival minus clock-to-Q, over 8 stages). An 8-bit multiplier would then have a data arrival of about $0.398 + 16 \times 0.281 = 4.90$ ns and, with the 0.135 ns setup time, $T_{\min} \approx 5.04$ ns ($F_{\max} \approx 199$ MHz), 1.81 times the 4-bit minimum period. This is an analytical estimate, not a simulation result. It assumes that synthesis maps each full-adder stage as efficiently as in the 4-bit design.

8 Conclusion

A 4-bit array multiplier with registered inputs and outputs was taken from Verilog RTL to a verified GDSII layout on SKY130, using only open-source tools.

- **Part A.** The RTL implements 16 AND gates, 12 full adders and 16 flip-flops exactly as specified. The exhaustive testbench passes all 256 vectors, detects an injected full-adder fault (FAIL: 256 vectors checked, 95 errors) and also checks the 2-cycle latency.
- **Part B.** Yosys maps the design to 61 cells (756.976 µm^2), 52.89 % of which is flip-flop area. The gate-level netlist passes the same testbench.
- **Part C.** At the TT corner, $T_{\min} = 2.79$ ns and $F_{\max} = 358.42$ MHz. The register-to-register critical path `a_r[1]`→`p[6]` crosses eight full-adder stages, matching the $2N$ estimate.

- **Part D.** LibreLane produces a DRC- and LVS-clean layout at 3.5 ns with +0.5188 ns TT setup slack, a post-route $F_{\max}$ of 335.44 MHz, 1873 cell instances and 2610 µm of routing.

9 Individual Contributions

We did the assignment together. Rather than splitting it into parts, we worked side by side on every part, as is expected of a team assignment, which is why most commits are co-authored. We went through the results together: the cell and area report, the critical path, the corner behaviour, and the choice of the 3.5 ns run as the final Part D result. We wrote and reviewed the README and this report together.

References

[1] *Digital VLSI Design: 4-bit Array Multiplier from RTL to GDSII with Open-Source Tools*, assignment specification (`RTL_to_GDS.pdf`).

[2] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson, ch. 11 (datapath subsystems).

[3] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson.

[4] IIC-OSIC-TOOLS, https://github.com/iic-jku/IIC-OSIC-TOOLS.

[5] Icarus Verilog, https://steveicarus.github.io/iverilog.

[6] Yosys Open SYnthesis Suite, https://yosyshq.readthedocs.io/projects/yosys.

[7] OpenSTA, https://github.com/parallaxsw/OpenSTA.

[8] LibreLane, https://github.com/librelane/librelane.

[9] OpenROAD, https://openroad.readthedocs.io.

[10] Magic VLSI layout tool and Netgen, http://opencircuitdesign.com.

[11] KLayout, https://www.klayout.de.

[12] SkyWater SKY130 PDK, https://skywater-pdk.readthedocs.io.